

Project Documentation – EduGenie: Google Gemini Powered Learning Assistant

Project Description

EduGenie harnesses Google Gemini generative AI to give learners a fast, personalised study experience. The platform includes modules that explain concepts in simple language, generate practice quizzes and produce concise summaries / study plans. Built with a FastAPI backend and a web interface, EduGenie sends carefully engineered prompts to the Gemini API and displays structured results, with secure API key management through a .env file.

Scenarios

Scenario 1: A student enters "Explain binary search trees" and receives a step-by-step beginner-friendly explanation with an example.

Scenario 2: A learner pastes class notes and receives a short revision summary and five practice questions.

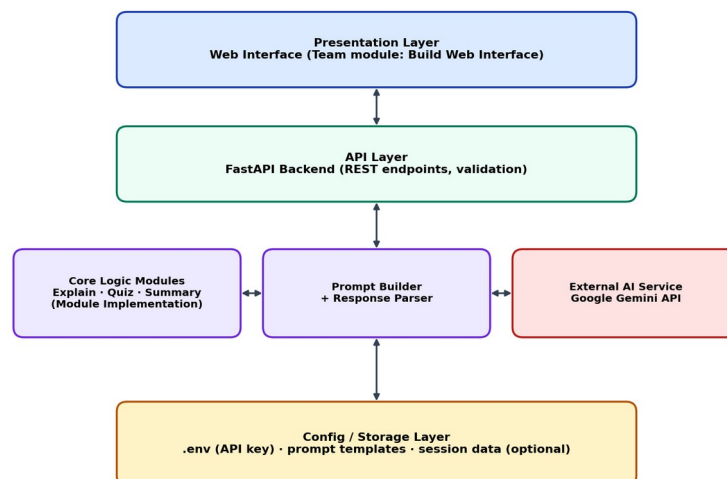
Scenario 3: A user preparing for an exam in two weeks requests a day-wise study plan for a subject.

Scenario 4 (Ask a Question): A student types a doubt such as "What is the difference between a stack and a queue?" and receives a clear, simple answer with a short real-life example, such as a pile of plates versus a line at a ticket counter.

Scenario 5 (Quiz): A learner selects the topic "Python Loops" and a difficulty level, and EduGenie generates a quiz of three multiple-choice questions. After the learner submits their answers, it shows the score, shows the correct answers.

Scenario 6 (Diagram / Flowchart): A student asks "Show the flowchart of the bubble sort algorithm" and receives a step-by-step flowchart (or a text-based diagram) showing the comparison and swap steps, which makes the process easy to visualize.

Technical Architecture



EduGenie follows a modular layered architecture: a web interface (presentation), a FastAPI backend that validates requests and orchestrates the learning modules, and the Google Gemini API as the generative AI engine. Configuration such as the API key lives in a .env file.

Pre-requisites

- FastAPI framework knowledge (FastAPI documentation)
- Google Gemini API familiarity (Google AI Studio / Gemini API docs)
- HTML, CSS and JavaScript skills
- Python programming proficiency
- Version control with Git and GitHub
- Development environment setup (Python 3.9+, pip, venv)

Project Workflow

Milestone / Epic	Activities
Milestone 1: Model Selection and Architecture	1.1 Generate a Google Gemini API key and store it securely 1.2 Select the appropriate Gemini model 1.3 Define application architecture 1.4 Set up the development environment
Milestone 2: Core Functionalities Development	2.1 Module Implementation (Krishnapriya P) 2.2 Backend API with FastAPI (Bhanumathi S)
Milestone 3: Frontend Development	3.1 Build Web Interface (Charumathi S) 3.2 Live Integration of frontend and backend (Bhanumathi S)
Milestone 4: Testing and Deployment	4.1 Run Locally (Jeeva S) 4.2 Functional Testing (Hari Babu S)
Milestone 5: Conclusion	5.1 Future Enhancements (Charumathi S)

Match the milestone names to your SkillWallet epics (7 epics in total).

Milestone 1: Model Selection and Architecture

Activity 1.1 – Generate a Gemini API Key

- Open Google AI Studio (aistudio.google.com) and sign in with your Google account.
- Click "Get API key" and then "Create API key".
- Copy the key immediately and store it safely.
- Create a .env file in the project root and add the key (below).
- Add .env to .gitignore so the key is never committed.

```
GEMINI_API_KEY=your_copied_api_key_here
```

Activity 1.2 – Select the Model:

Google Gemini Flash — primarily configured through GEMINI_MODEL, with fallback models

1. gemini-3.5-flash
2. gemini-2.5-flash
3. gemini-3.5-flash-lite
4. gemini-2.5-flash-lite
5. gemini-flash-latest

Activity 1.3 – Define the Architecture: Frontend collects input → FastAPI validates and builds the prompt → Gemini generates content → structured response is displayed.

Activity 1.4 – Set Up the Environment: Install Python and pip, create a virtual environment and install dependencies.

```
python -m venv venv
venv\Scripts\activate
pip install fastapi uvicorn google-generativeai python-dotenv
```

Milestone 2: Core Functionalities Development

Activity 2.1 – Module Implementation (Krishnapriya P): Implement the learning modules (ask questions, explain, quiz, summary, recommend learning path, create diagram/flowchart). Each module builds a task-specific prompt, calls Gemini and returns structured text.

Activity 2.2 – Backend API with FastAPI (Bhanumathi S): Define routes, validate input with Pydantic models, call the module functions and return JSON responses with proper error codes.

```
@app.post("/explain")
```

Milestone 3: Frontend Development

Activity 3.1 – Build Web Interface (Charumathi S): Develop a responsive single-page interface with an input area, module selection and results section.

Activity 3.2 – Live Integration (Bhanumathi S): Connect the interface to the FastAPI endpoints using fetch() so results appear live without page reloads; show loading and error states.

Milestone 4: Testing and Deployment

Activity 4.1 – Run Locally (Jeeva S): Install dependencies, set the API key and start the server with Uvicorn; verify the app loads in the browser.

Activity 4.2 – Functional Testing (Hari Babu S): Test each module with valid, empty and long inputs; verify error handling and UI behaviour.

Test Case	Input	Expected Result	Actual / Status
TC-1	Valid topic	Explanation displayed	Pass
TC-2	Empty input	Validation error shown	Pass
TC-3	Quiz request	Questions displayed	Pass
TC-4	API key missing/invalid	Friendly error message	Pass

Milestone 5: Conclusion

EduGenie demonstrates how Google Gemini can power an accessible learning assistant. Future Enhancements (Charumathi S) include user accounts, saved history, multi-language and voice support, and cloud deployment (see 8.4).

Output Screenshots

Screenshots showing the EduGenie webpage, including task selection options such as Ask Question, Explain Concept, Quiz Generation, Summary, Recommend learning path and Create diagram/flowchart.

Its outputs are provided here: https://drive.google.com/drive/folders/1tf560_-bOhC_Yo9-pwIx7iQ5vIPvvcV5